

CENTRO UNIVERSITÁRIO - CATÓLICA DE SANTA CATARINA
ENGENHARIA DE SOFTWARE
PROJETO DE APRENDIZAGEM COLABORATIVA

MARUAN BIASI EL ACHKAR

**AGILIS - LEITOR E INTERPRETADOR DE LICITAÇÕES COM INTELIGÊNCIA
ARTIFICIAL**

JOINVILLE
2025

RESUMO

O projeto Agilis Licitações propõe um sistema automatizado capaz de interpretar e extrair informações essenciais de editais de licitações farmacêuticas voltadas à aquisição de medicamentos. A solução utiliza uma combinação sequencial de algoritmos de lógica, expressões regulares, aprendizado de máquina e outras técnicas de inteligência artificial, permitindo refinar progressivamente os resultados e alcançar elevado grau de precisão. Dessa forma, o sistema reduz drasticamente o tempo necessário para a análise desses documentos, atualmente realizada de forma manual e suscetível a erros. Além de aumentar a competitividade entre empresas, o Agilis contribui com a democratização do acesso ao mercado de compras públicas, tornando o processo de participação em licitações mais ágil, preciso e eficiente.

Palavras-chave: Licitações. Inteligência Artificial. Automação. Processamento de Editais. Análise de Documentos.

SUMÁRIO

1	INTRODUÇÃO	4
1.1	CONTEXTO	4
1.2	JUSTIFICATIVA	4
1.3	OBJETIVOS	5
2	DESCRIÇÃO DO PROJETO	6
2.1	LINHA DE PROJETO	6
2.2	TEMA DO PROJETO	6
2.3	PROPÓSITO E USO PRÁTICO	6
2.4	PÚBLICO-ALVO	6
2.5	PROBLEMAS A RESOLVER	6
2.6	DIFERENCIAÇÃO/INEDITISMO	7
2.7	LIMITAÇÕES	7
2.8	NORMAS E LEGISLAÇÕES APLICÁVEIS	7
2.9	MÉTRICAS DE SUCESSO	7
3	ESPECIFICAÇÕES TÉCNICAS	9
3.1	REQUISITOS DE SOFTWARE	9
3.1.1	Requisitos Funcionais (RF)	9
3.1.2	Requisitos Não-Funcionais (RNF)	9
3.1.3	Representação dos Requisitos	10
3.1.4	Aderência aos Requisitos da Linha de Projeto	10
3.1.5	Ambiente de Treinamento e Conjunto de Dados	11
3.2	PIPELINE DE ALGORITMOS DE IA	11
3.3	CONSIDERAÇÕES DE DESIGN	13
3.3.1	Visão Inicial da Arquitetura	13
3.3.2	Padrões de Arquitetura	14
3.3.3	Modelo Entidade Relacionamento	14
3.3.4	Diagramas	15
3.3.4.0.1	C4 — Nível 1 (Contexto)	15
3.3.4.0.2	C4 — Nível 2 (Containers)	15
3.3.4.0.3	C4 — Nível 3 (Componentes do Backend)	15
3.3.5	Mockups das Telas Principais	15
3.3.5.0.1	Tela de Login	15
3.3.5.0.2	Tela de Análise em Execução	15
3.3.5.0.3	Tela de Resultados	15
3.3.6	Decisões e Alternativas Consideradas	19

3.3.7	Critérios de Escalabilidade, Resiliência e Segurança	19
3.4	FERRAMENTAS TECNOLÓGICAS	20
3.4.1	Linguagens de Programação	20
3.4.2	Frameworks e Bibliotecas	20
3.4.3	Ferramentas de Desenvolvimento e Gestão	21
3.4.4	Licenciamento	21
3.5	CONSIDERAÇÕES DE SEGURANÇA	22
3.5.1	Riscos Identificados	22
3.5.2	Medidas de Mitigação	22
3.5.3	Normas e Boas Práticas Seguidas	22
3.5.4	Responsabilidade Ética	23
	REFERÊNCIAS	24
	APÊNDICE A – CONJUNTO DE LICITAÇÕES UTILIZADAS NA	
	AVALIAÇÃO	27

1 INTRODUÇÃO

O avanço das técnicas de inteligência artificial e do processamento de linguagem natural (NLP) tem permitido o desenvolvimento de sistemas capazes de interpretar automaticamente textos complexos. Esses avanços são impulsionados por modelos cada vez mais poderosos de aprendizagem profunda, que expandem as fronteiras do entendimento computacional de linguagem humana (Russell; Norvig, 2020; Jurafsky; Martin, 2023; Goodfellow; Bengio; Courville, 2016). Tecnologias baseadas em NLP têm ganhado espaço em diversos setores, especialmente em áreas que dependem de análise documental intensiva.

No âmbito das licitações públicas, surge a oportunidade de aplicar métodos computacionais avançados para modernizar um processo que ainda depende fortemente da leitura manual. Relatórios recentes apontam a falta de padronização e a complexidade estrutural dos editais, dificultando sua interpretação ((TCU), 2022; Brasil, 2021). O *Agilis Licitações* se insere nesse contexto como uma solução que utiliza algoritmos de lógica, expressões regulares e modelos de aprendizado de máquina para apoiar a compreensão estruturada de editais.

1.1 CONTEXTO

Editais de licitações públicas são documentos longos, heterogêneos e sem padronização clara entre diferentes órgãos, apesar das diretrizes gerais estabelecidas pela Nova Lei de Licitações (Brasil, 2021). A linguagem jurídica, os múltiplos anexos e a variação na estrutura entre instituições tornam o processo de leitura complexo e exigente, como apontado também por auditorias de conformidade do Tribunal de Contas da União ((TCU), 2022).

Esses fatores também limitam significativamente a capacidade de algoritmos tradicionais baseados apenas em regras fixas ou buscas textuais simples, cuja performance diminui em cenários altamente variáveis (Kim; Lee; Park, 2019). Como consequência, a extração de informações essenciais ainda é majoritariamente realizada de forma manual, dependente de profissionais especializados e sujeita a atrasos e inconsistências.

Nesse cenário, soluções baseadas em inteligência artificial tornam-se particularmente relevantes. Avanços recentes em modelos de linguagem, como BERT e suas variações jurídicas (Devlin et al., 2019; Chalkidis et al., 2020), permitem automatizar tarefas antes consideradas exclusivamente humanas, como interpretar estruturas textuais irregulares e identificar padrões semânticos em documentos complexos.

1.2 JUSTIFICATIVA

O *Agilis Licitações* é relevante para a engenharia de software por demonstrar como técnicas modernas de inteligência artificial podem automatizar tarefas predominantemente dependentes de trabalho humano, especialmente em domínios onde a interpretação textual é complexa e exige alto grau de precisão. A área jurídica, em especial, tem se beneficiado

de modelos especializados de análise textual (Chalkidis et al., 2020; Hendrycks et al., 2020), mostrando o potencial da IA para lidar com documentos estruturados e não estruturados.

Sua importância reside na capacidade de transformar um processo manual e demorado em um fluxo automatizado, reduzindo erros, aumentando a velocidade e ampliando o acesso de empresas às oportunidades de compras públicas. Dados recentes indicam a abertura de milhares de novas licitações mensalmente no país (Públicas, 2024), reforçando a necessidade de ferramentas computacionais que democratizem o acesso às informações e aumentem a competitividade. Assim, o projeto resolve um problema concreto: a dificuldade de extrair informações essenciais de editais de forma rápida, precisa e padronizada, contribuindo para a evolução de soluções inteligentes dentro da engenharia de software.

1.3 OBJETIVOS

- **Objetivo Principal:** Desenvolver um sistema capaz de ler, interpretar e extrair automaticamente informações essenciais de editais de licitação, reduzindo o tempo de análise e aumentando a eficiência e competitividade das empresas participantes.
- **Objetivos Secundários:**
 - Desenvolver a interface do usuário para visualização e interação com os resultados das análises;
 - Implementar o banco de dados para armazenamento estruturado das informações extraídas;
 - Implementar mecanismos de hashing para evitar o reprocessamento de arquivos duplicados;
 - Desenvolver o sistema de login e autenticação de usuários;
 - Implementar a persistência de dados no backend;
 - Criar o histórico de análises e resultados anteriores acessível aos usuários.
 - Realizar o tratamento dos editais, composto por extração de texto, limpeza de texto, normalização de caracteres e segmentação do edital.
 - Executar o fine tuning e treinamento complementar de algoritmos e modelos de inteligência artificial já existentes, também realizando os ajustes necessários para esse projeto.

2 DESCRIÇÃO DO PROJETO

2.1 LINHA DE PROJETO

O projeto *Agilis Licitações* se enquadra na categoria **Projetos com Inteligência Artificial (IA)**, alinhando-se às diretrizes atuais de soluções inteligentes para automação de processos complexos (Russell; Norvig, 2020).

2.2 TEMA DO PROJETO

Desenvolvimento de um sistema inteligente composto por múltiplos algoritmos executados de forma sequencial para ler, interpretar e extrair automaticamente informações essenciais de editais de licitações públicas. Essa abordagem acompanha o avanço recente de modelos especializados para processamento de documentos jurídicos, como o *Legal-BERT* (Chalkidis et al., 2020).

2.3 PROPÓSITO E USO PRÁTICO

O propósito do *Agilis Licitações* é automatizar o processo de leitura e interpretação de editais de licitação, substituindo a análise manual por uma análise automática estruturada. Na prática, o sistema processa os editais de uma licitação e gera uma lista organizada com informações essenciais, como prazos, exigências, documentos obrigatórios e itens licitados. Esse cenário é reforçado pela crescente necessidade de modernização das compras públicas em decorrência da elevada quantidade de editais publicados e da ausência de padronização ((TCU), 2022; Brasil, 2021).

2.4 PÚBLICO-ALVO

O público-alvo são empresas e profissionais que participam de licitações públicas, especialmente no setor farmacêutico. Dados recentes demonstram que esse segmento envolve grande volume de editais e alta complexidade documental (Públicas, 2024).

2.5 PROBLEMAS A RESOLVER

- O longo tempo necessário para leitura e interpretação de editais, devido à heterogeneidade estrutural e ausência de padronização ((TCU), 2022);
- A dificuldade de compreensão de textos jurídicos complexos, característica reconhecida em estudos sobre documentos legais (Chalkidis et al., 2020);
- A ocorrência de erros humanos durante a análise manual, especialmente em tarefas repetitivas e extensas (Kim; Lee; Park, 2019);

- A dificuldade em localizar informações essenciais para a participação, problema recorrente em editais longos e com estrutura variável ((TCU), 2022).

2.6 DIFERENCIAÇÃO/INEDITISMO

O principal diferencial do *Agilis Licitações* é o uso de múltiplos algoritmos executados de forma sequencial, integrando técnicas de lógica, busca textual, aprendizado de máquina e inteligência artificial. Essa abordagem é mais robusta do que métodos baseados apenas em regras fixas, que apresentam limitações quando aplicados a documentos heterogêneos (Kim; Lee; Park, 2019). A combinação de heurísticas e modelos de linguagem modernos permite resultados mais precisos e rápidos, alinhados às tendências observadas em soluções de NLP voltadas ao setor jurídico (Hendrycks et al., 2020).

2.7 LIMITAÇÕES

- Não substitui a decisão final humana sobre elegibilidade ou envio de propostas;
- Não realiza o envio automático de propostas;
- Não abrange editais fora do escopo farmacêutico oncológico, sendo essa limitação comum em sistemas especializados de NLP devido ao treinamento em domínios específicos (Chalkidis et al., 2020).

2.8 NORMAS E LEGISLAÇÕES APLICÁVEIS

O projeto observa algumas das diretrizes da Lei nº 13.709/2018 — Lei Geral de Proteção de Dados (LGPD) — especificamente no tratamento de credenciais de usuários, garantindo segurança, confidencialidade e armazenamento adequado de login e senha. Além disso, atende aos princípios da Nova Lei de Licitações (Lei nº 14.133/2021) no que se refere ao manuseio de documentos públicos (Brasil, 2021). Ressalta-se que os editais processados pelo sistema são documentos oficiais públicos, não envolvendo dados pessoais sensíveis, conforme entendimento dos órgãos de compras governamentais (Públicas, 2024).

2.9 MÉTRICAS DE SUCESSO

As métricas adotadas seguem práticas consolidadas em Machine Learning e NLP, fundamentadas em princípios gerais de avaliação de desempenho da área (Bishop, 2006). O *F1-score* é amplamente utilizado para avaliar tarefas de extração de informações (Manning; Schütze, 1999), enquanto métricas operacionais, como tempo de processamento e taxa de falhas, são recomendadas na avaliação de pipelines de ML (Lakshmanan; Robinson; Munn, 2020).

- Alcançar pelo menos 70% de *F1-score* médio na extração de informações essenciais dos editais, métrica que combina precisão e abrangência, garantindo que o sistema não deixe de extrair informações importantes nem gere informações incorretas (Manning; Schütze, 1999);
- Atingir um tempo médio de processamento inferior a 30 minutos por edital, incluindo etapas de pré-processamento, extração e geração do relatório final;
- Manter taxa de falhas operacionais inferior a 10%, considerando como falha qualquer evento que interrompa o processamento de um edital, incluindo erros inesperados, ou interrupções na pipeline que impeçam a conclusão da análise automática (Lakshmanan; Robinson; Munn, 2020; Breck et al., 2022).

3 ESPECIFICAÇÕES TÉCNICAS

3.1 REQUISITOS DE SOFTWARE

3.1.1 Requisitos Funcionais (RF)

Os requisitos funcionais do sistema foram definidos com base em boas práticas de engenharia de software e especificação de requisitos (Sommerville, 2011).

- **RF1:** O sistema deve permitir o upload de um ou múltiplos arquivos de edital em formato PDF ou texto;
- **RF2:** O sistema deve realizar a leitura automática dos arquivos enviados e identificar suas seções principais;
- **RF3:** O sistema deve interpretar e extrair informações essenciais, como prazos, documentos exigidos, itens licitados e critérios de participação;
- **RF4:** O sistema deve armazenar os resultados das análises em um banco de dados;
- **RF5:** O sistema deve oferecer uma interface visual para exibição dos resultados processados;
- **RF6:** O sistema deve manter um histórico de análises realizadas por cada usuário;
- **RF7:** O sistema deve permitir login e autenticação de usuários;
- **RF8:** O sistema deve evitar o reprocessamento de arquivos idênticos, aplicando hashing de verificação nos uploads.

3.1.2 Requisitos Não-Funcionais (RNF)

Os requisitos não funcionais complementam as funcionalidades descritas, especificando restrições de desempenho, qualidade, usabilidade e manutenção do sistema (Sommerville, 2011).

- O tempo médio de processamento por edital deve ser inferior a 30 minutos, avaliado em um conjunto de 5 licitações previamente selecionadas (ver Apêndice A);
- O desempenho do modelo de extração das informações essenciais deve alcançar pelo menos 70% de *F1-score* médio, avaliado nas mesmas 5 licitações previamente selecionadas (ver Apêndice A);
- O sistema deve apresentar estabilidade operacional mínima de 90%, sem falhas inesperadas, avaliada no processamento de 5 licitações previamente selecionadas (ver Apêndice A) e 5 licitações adicionais escolhidas aleatoriamente;

- As credenciais de usuário devem ser armazenadas com hashing criptográfico;
- A interface deve ser intuitiva, responsiva e de fácil uso;
- O código deve seguir boas práticas de modularidade e escalabilidade para permitir evolução futura.

3.1.3 Representação dos Requisitos

A Figura 1 apresenta o diagrama de casos de uso do sistema *Agilis Licitações*, destacando as principais interações do usuário com o sistema, conforme a abordagem de modelagem de requisitos baseada em casos de uso (Fowler, 2004).

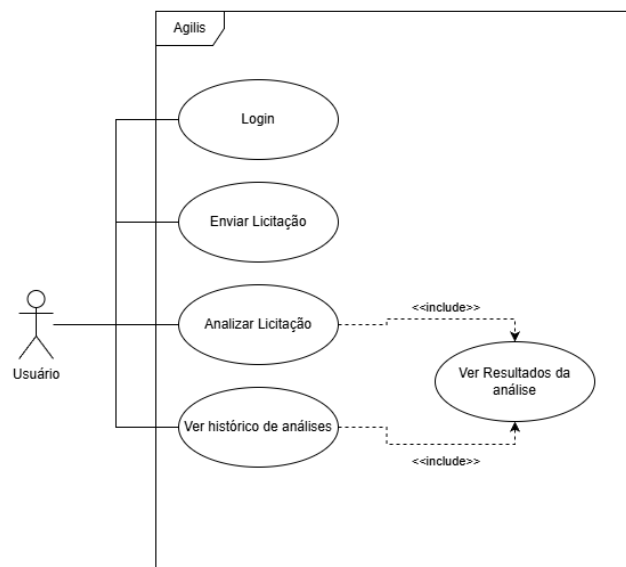


Figura 1 – Diagrama de casos de uso do sistema *Agilis Licitações*.

Fonte: Elaborado pelo autor (2025).

3.1.4 Aderência aos Requisitos da Linha de Projeto

O projeto *Agilis Licitações* atende aos critérios obrigatórios da categoria **Projetos com Inteligência Artificial (IA)** pelos seguintes aspectos:

- Emprega múltiplos algoritmos de inteligência artificial para processamento de texto, alinhados à evolução recente de modelos de linguagem (Devlin et al., 2019; Chalkidis et al., 2020);
- Aplica técnicas de *machine learning* e NLP na análise de documentos jurídicos (Hendrycks et al., 2020);
- Utiliza arquitetura híbrida com Node.js e Python FastAPI, integrando backend com módulos inteligentes (Node.js Foundation, 2024; Tiangolo, 2024);

- Define métricas de acurácia, desempenho e estabilidade para avaliação do modelo de IA.

3.1.5 Ambiente de Treinamento e Conjunto de Dados

O treinamento dos modelos de inteligência artificial do *Agilis Licitações* será realizado majoritariamente em ambiente local, utilizando a estação de trabalho do autor para experimentos iniciais, ajustes de hiperparâmetros e testes de menor porte. Para treinamentos mais pesados, que demandem maior capacidade computacional, será utilizado o *Google Colab Pro*, aproveitando recursos de GPU em nuvem. Em casos específicos que exijam processamento intensivo ou execução em larga escala, está previsto o uso de infraestrutura em nuvem na *Amazon Web Services (AWS)*, de forma pontual, quando necessário.

O conjunto de treino será composto por editais públicos de licitação e por uma base de dados anotada manualmente, contendo as informações essenciais extraídas desses documentos (prazos, documentos exigidos, itens licitados, critérios de participação, entre outros). Essas anotações serão fornecidas pela empresa *Amoveri Farma S.a.*, com base em licitações passadas que a empresa já analisou. O objetivo é alcançar um conjunto com aproximadamente **1.000 editais** anotados, estabelecendo como mínimo viável uma base de **300 editais** para os primeiros experimentos.

Todos os editais utilizados são documentos públicos, conforme estabelecido pela Lei nº 14.133/2021, que determina a obrigatoriedade de publicidade e divulgação eletrônica dos editais de licitação em plataformas oficiais como o PNCP (Brasil, 2021). As informações anotadas referem-se apenas a requisitos e regras de participação em licitações já encerradas, não contendo dados pessoais sensíveis. Dessa forma, o conjunto de treino não envolve tratamento de informações sigilosas ou protegidas pela LGPD, mantendo o foco apenas em dados jurídicos e administrativos de domínio público.

3.2 PIPELINE DE ALGORITMOS DE IA

O núcleo inteligente do *Agilis Licitações* é estruturado como uma *pipeline* sequencial de algoritmos, no qual cada etapa refina o resultado da anterior, desde a leitura do edital em PDF até a geração de uma saída estruturada em formato JSON. Esse encadeamento permite combinar regras determinísticas, técnicas clássicas de processamento de texto e modelos de linguagem modernos, explorando o melhor de cada abordagem em termos de precisão, desempenho e robustez.

1. Ingestão e normalização do texto

Inicialmente, os arquivos de edital enviados pelo usuário (pode ser apenas um ou múltiplos PDFs) são unificados em um único documento lógico, quando necessário. Em seguida, bibliotecas especializadas como *pdfplumber* e *PyMuPDF* são utilizadas para extrair o texto bruto dos PDFs, de acordo com as práticas clássicas de parsing de documentos PDF

(Hassan; Baumgartner, 2016), descartando elementos gráficos e mantendo apenas o conteúdo textual relevante. Sobre esse texto é aplicado um conjunto de regras de normalização, incluindo a padronização de quebras de linha, remoção de espaçamentos irregulares e recomposição de linhas que foram quebradas no meio de frases, resultando em um texto contínuo e adequado para processamento posterior.

2. Segmentação inteligente em seções

Para contornar limitações de *context window* de modelos de linguagem (Beltagy; Peters; Cohan, 2020) e evitar o processamento de documentos extremamente extensos como um único bloco, o sistema realiza a segmentação do edital em seções coerentes (Utiyama; Isahara, 2001). Essa etapa combina heurísticas baseadas em expressões regulares e padrões típicos de editais (títulos em caixa alta, numeração hierárquica, expressões como “DO OBJETO”, “DAS CONDIÇÕES DE PARTICIPAÇÃO” etc.) com um classificador leve baseado em modelos do tipo *BERT/LegalBERT* (Devlin et al., 2019; Chalkidis et al., 2020). As heurísticas identificam candidatos a início de seção e o classificador supervisionado decide, para cada candidato, se há de fato uma fronteira de seção, reduzindo o risco de cortar informações importantes ou fragmentar cláusulas relacionadas.

3. Localização de trechos candidatos

Com o edital segmentado em seções menores, o sistema precisa localizar onde, dentro do texto, estão as informações de interesse (prazos, documentos exigidos, itens licitados, critérios de participação etc.). Essa etapa começa com um módulo de **dicionário de termos**, que busca palavras-chave e expressões jurídicas ou técnicas pré-cadastradas, marcando seções potencialmente relevantes para cada tipo de informação. Em seguida, um módulo de **busca semântica com *embeddings*** transforma parágrafos em vetores e compara esses vetores com consultas semânticas pré-definidas (por exemplo, “prazo para apresentação das propostas”, “documentos necessários para habilitação”). Um algoritmo de ranqueamento combina os resultados do dicionário e da busca semântica, produzindo um conjunto reduzido de trechos candidatos para cada categoria de informação.

4. Extração fina das informações

A etapa seguinte é responsável por extrair, de cada trecho candidato, os valores específicos que serão apresentados ao usuário (datas, listas de documentos, descrições de itens, percentuais, valores, entre outros). Para isso, o sistema utiliza um modelo de extração baseado em transformadores com *fine tuning* supervisionado, seguindo a arquitetura e práticas descritas por (Wolf et al., 2020), treinado sobre o conjunto de editais anotados manualmente pela empresa *Amoveri Farma S.a.*. Esse modelo recebe como entrada o trecho candidato e o tipo de informação a ser extraída, retornando os *spans* de texto mais prováveis que contenham a resposta. Em paralelo, um módulo complementar baseado em expressões regulares é empregado para identificar e normalizar padrões específicos,

como datas, valores monetários, CNPJ e números de processo, garantindo consistência de formato e maior confiança nos dados estruturados.

5. Consolidação e organização dos resultados

Por fim, os resultados extraídos passam por uma etapa de consolidação, na qual são combinados scores de confiança do modelo de extração, sinais da busca semântica e presença de termos do dicionário, a fim de selecionar os melhores valores por campo e resolver possíveis ambiguidades. Em seguida, um módulo leve baseado em modelo de linguagem (*LLM*) é utilizado apenas para organizar as informações em uma estrutura JSON padronizada e, opcionalmente, gerar resumos explicativos para apresentação no frontend. O uso desse LLM é inspirado no papel de modelos de linguagem de grande porte na formatação e organização textual, conforme demonstrado em (Brown et al., 2020). O LLM opera sobre os dados já extraídos, não como fonte primária de decisão, reduzindo o risco de alucinações e garantindo que o sistema permaneça fiel ao conteúdo original dos editais.

Essa sequência de algoritmos forma um fluxo de processamento completo, onde cada módulo tem uma responsabilidade clara e contribui para refinar a interpretação dos editais.



Figura 2 – Pipeline de algoritmos do sistema *Agilis Licitações*.

Fonte: Elaborado pelo autor (2025).

3.3 CONSIDERAÇÕES DE DESIGN

3.3.1 Visão Inicial da Arquitetura

O sistema segue uma arquitetura modular em camadas, composta pelos seguintes componentes principais, em conformidade com princípios de arquitetura de software orientada a responsabilidade e separação de camadas (Bass; Clements; Kazman, 2012):

- **Interface Visual (Frontend — React):** permite o upload de arquivos, o login de usuários e a visualização dos resultados das análises (React Contributors, 2024);
- **Servidor Principal (Backend — Express.js):** gerencia autenticação, requisições, armazenamento de dados e a comunicação com o módulo de IA (Node.js Foundation, 2024);
- **Módulo de IA (FastAPI — Python):** executa os algoritmos de leitura, extração textual, análise, lógica, expressões regulares, NLP e aprendizado de máquina (Tiangolo, 2024);

- **Banco de Dados (PostgreSQL):** armazena usuários, logs, resultados de análises e hashes de verificação para evitar reprocessamentos (PostgreSQL Global Development Group, 2024).

3.3.2 Padrões de Arquitetura

O sistema adota uma **Arquitetura em Camadas**, separando responsabilidades de forma clara entre os módulos. Esse padrão facilita a manutenção, a escalabilidade e a organização do código, além de permitir a evolução independente de cada componente (Bass; Clements; Kazman, 2012).

As camadas utilizadas são:

- **Camada de Apresentação (Frontend — React):** responsável pela interação com o usuário, envio de arquivos e exibição dos resultados;
- **Camada de Aplicação (Backend — Express.js):** coordena o fluxo de requisições, autenticação, regras de negócio e comunicação com o módulo de IA;
- **Camada de Serviço de IA (FastAPI — Python):** executa os algoritmos de análise textual, NLP, lógica e aprendizado de máquina;
- **Camada de Dados (PostgreSQL):** armazena usuários, resultados de análises, logs e informações persistentes do sistema.

3.3.3 Modelo Entidade-Relacionamento

A Figura 3 apresenta o Modelo Entidade-Relacionamento (MER) do sistema *Agilis Licitações*, descrevendo as entidades principais e suas relações. O modelo inclui as entidades **Usuário**, **Edital** e **Análise**, permitindo registrar quem realizou cada análise, qual edital foi processado e os resultados obtidos.

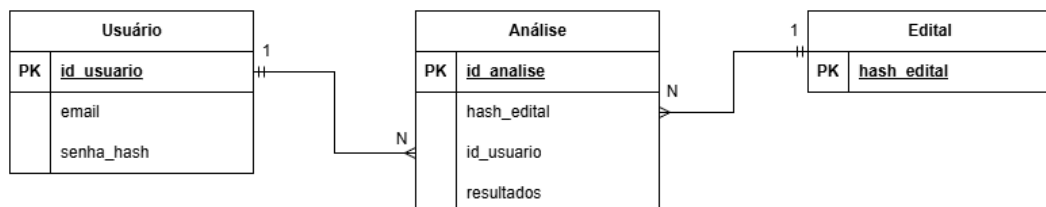


Figura 3 – Modelo Entidade-Relacionamento do sistema *Agilis Licitações*.

Fonte: Elaborado pelo autor (2025).

3.3.4 Diagramas

Os diagramas desta seção representam a arquitetura do sistema utilizando o *C4 Model* (Brown, 2018) nos níveis de Contexto, Containers e Componentes, além do Modelo Entidade-Relacionamento apresentado anteriormente. Os diagramas foram produzidos com o objetivo de facilitar o entendimento estrutural e operacional do *Agilis Licitações*.

3.3.4.0.1 C4 — Nível 1 (Contexto)

A Figura 4 apresenta a visão de contexto do sistema, destacando a interação entre o usuário e o *Agilis Licitações*.

3.3.4.0.2 C4 — Nível 2 (Containers)

A Figura 5 apresenta os containers do sistema, incluindo frontend, backend, módulo de IA e banco de dados, com suas responsabilidades e fluxos de comunicação.

3.3.4.0.3 C4 — Nível 3 (Componentes do Backend)

A Figura 6 detalha os principais componentes internos do *Backend Express*, incluindo controladores, serviços auxiliares e comunicação com o módulo de IA e o banco de dados.

3.3.5 Mockups das Telas Principais

As principais telas do sistema foram prototipadas para representar o fluxo de uso previsto no *Agilis Licitações*. Os mockups a seguir demonstram as interfaces de acesso, análise e visualização dos resultados.

3.3.5.0.1 Tela de Login

A Figura 7 apresenta a tela de login do sistema, onde o usuário informa suas credenciais de acesso para entrar na plataforma e iniciar as análises.

3.3.5.0.2 Tela de Análise em Execução

A Figura 8 mostra o estado de processamento do edital, indicando ao usuário que o sistema está executando os algoritmos de análise.

3.3.5.0.3 Tela de Resultados

A Figura 9 apresenta a tela onde o usuário visualiza o edital enviado e os resultados extraídos automaticamente, incluindo critérios de habilitação, prazos, riscos e recomendações.



Figura 4 – C4 Model — Nível 1 (Contexto) do sistema *Agilis Licitações*.

Fonte: Elaborado pelo autor (2025).

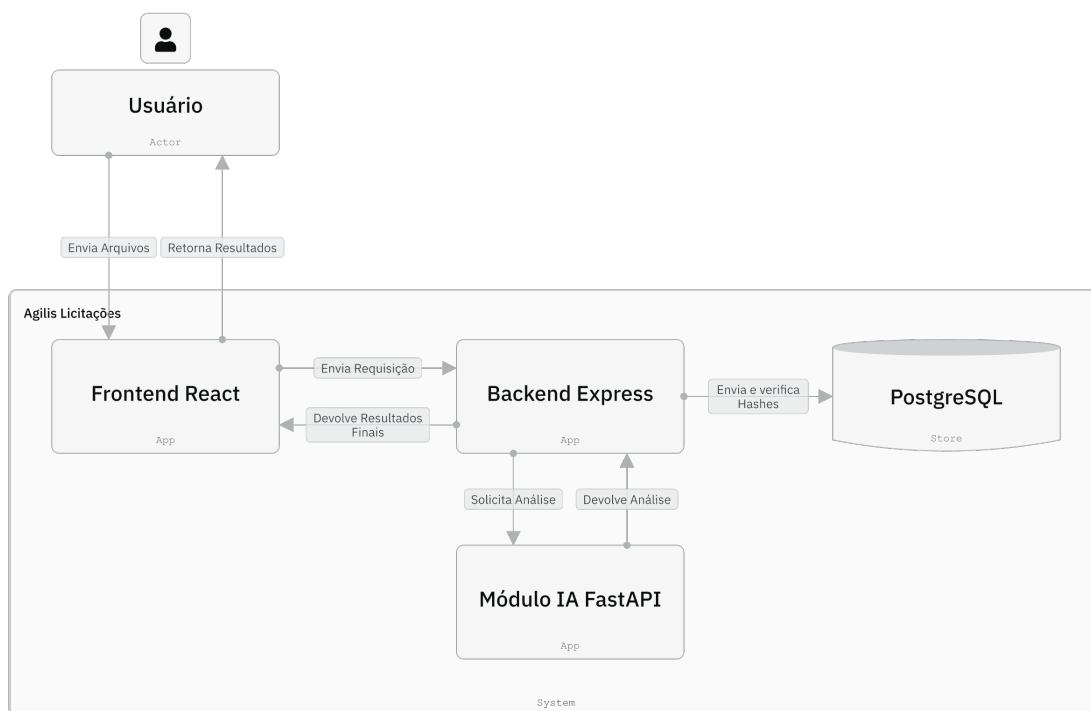


Figura 5 – C4 Model — Nível 2 (Containers) do sistema *Agilis Licitações*.

Fonte: Elaborado pelo autor (2025).

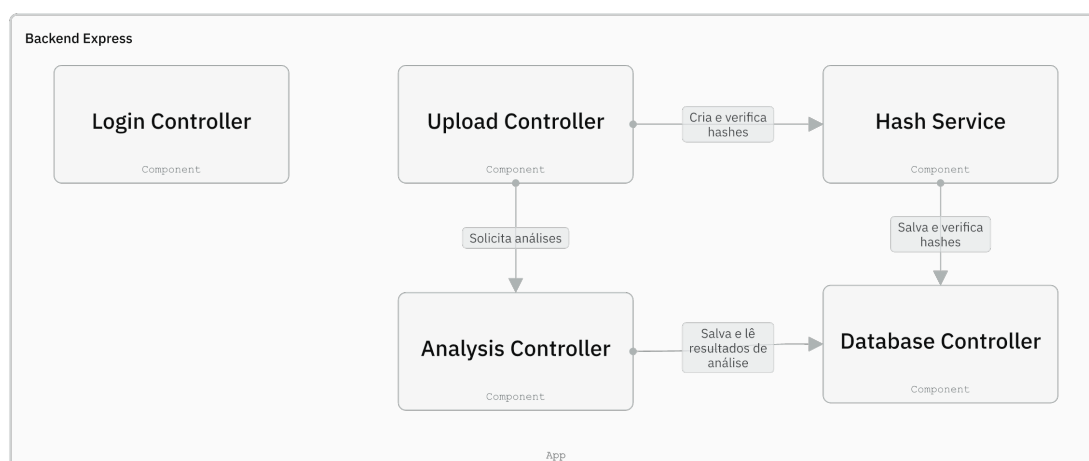


Figura 6 – C4 Model — Nível 3 (Componentes do Backend Express).

Fonte: Elaborado pelo autor (2025).

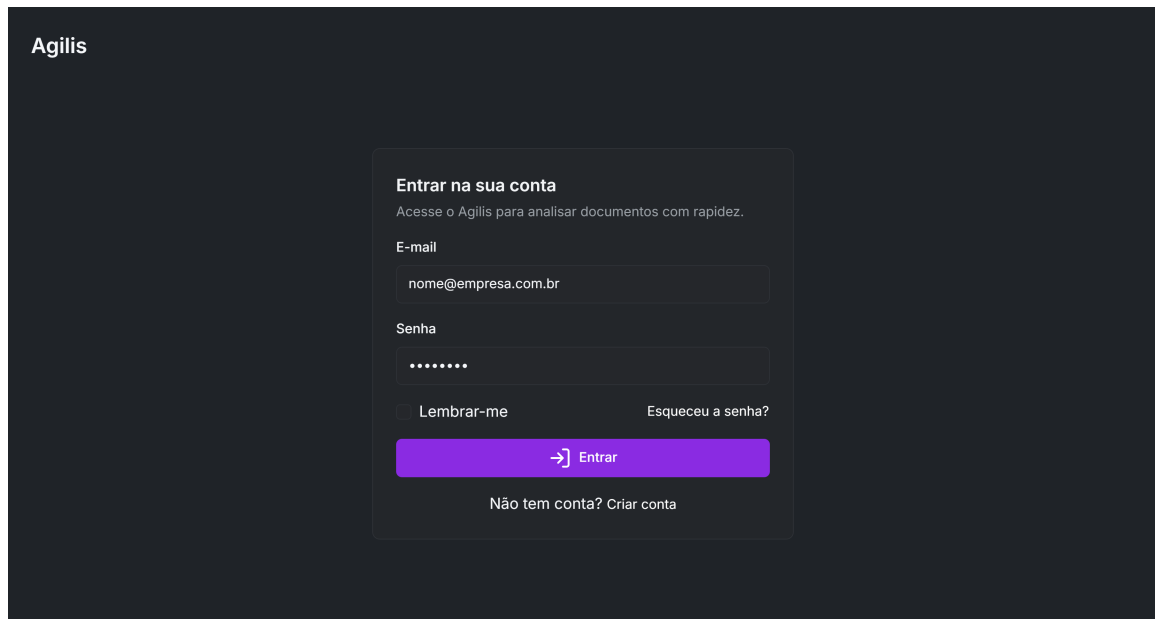


Figura 7 – Mockup da tela de login do sistema *Agilis Licitações*.

Fonte: Elaborado pelo autor (2025).

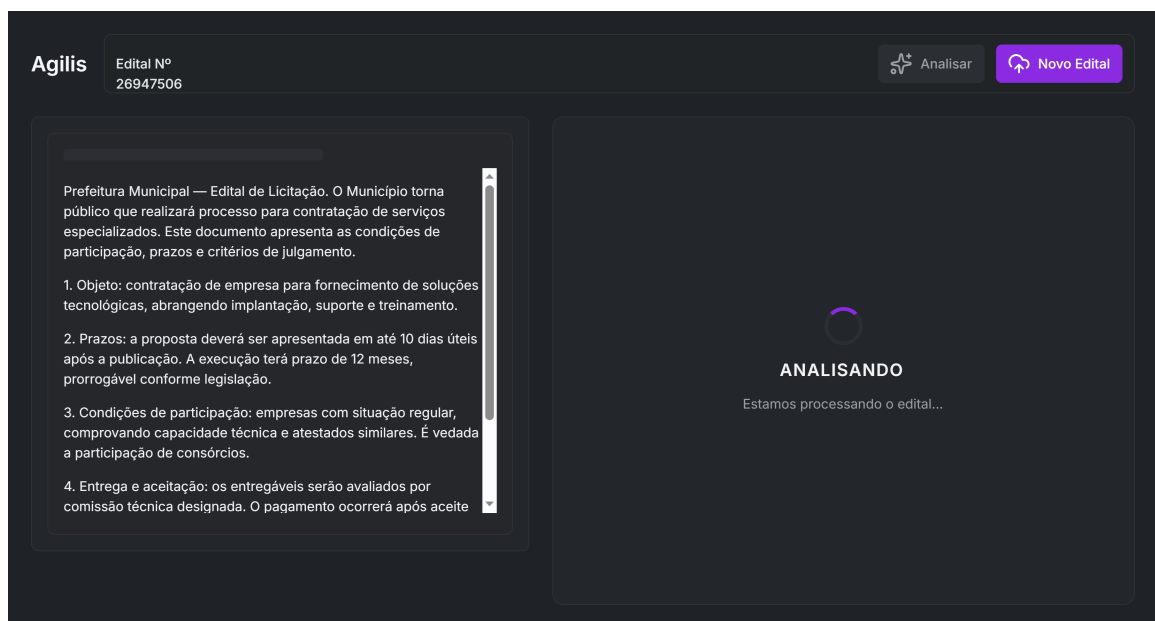


Figura 8 – Mockup da tela de análise em execução.

Fonte: Elaborado pelo autor (2025).

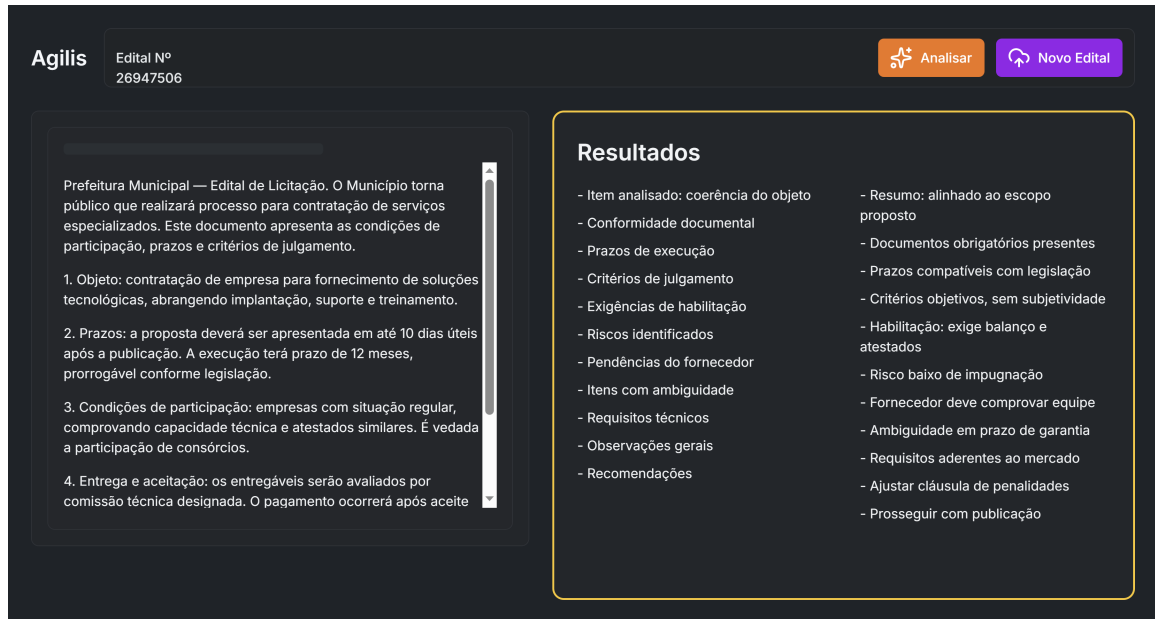


Figura 9 – Mockup da tela de visualização dos resultados da análise.

Fonte: Elaborado pelo autor (2025).

3.3.6 Decisões e Alternativas Consideradas

A arquitetura híbrida adotada combina Node.js no backend principal e Python (FastAPI) no módulo de IA. Essa escolha foi motivada pela eficiência do Node.js em operações assíncronas, gerenciamento de requisições e integração com o frontend (Node.js Foundation, 2024), enquanto o Python oferece robustez e maturidade em bibliotecas de processamento de linguagem natural, aprendizado de máquina e análise textual (Python Software Foundation, 2024; Pandas Development Team, 2024). No frontend, o uso de React e bibliotecas de componentes como *shadcn/ui* favorece produtividade e consistência visual (React Contributors, 2024; shadcn, 2024).

Como alternativa, foi avaliada a construção de todo o sistema em uma única linguagem, utilizando frameworks Python como Django ou Flask. No entanto, essa abordagem foi descartada devido à menor flexibilidade em cenários de alto volume de requisições assíncronas e à integração menos fluida com o ecossistema moderno de desenvolvimento web. A separação dos módulos permitiu maior desempenho, modularidade e independência tecnológica entre as camadas (Bass; Clements; Kazman, 2012).

3.3.7 Critérios de Escalabilidade, Resiliência e Segurança

Os critérios adotados visam garantir que o sistema possa crescer, lidar com falhas e manter a segurança dos dados dos usuários e dos processos de análise.

- **Escalabilidade:** o sistema é hospedado em um servidor centralizado, permitindo acesso remoto simples e possibilidade de expansão futura por meio de réplicas do backend ou do módulo de IA, caso haja aumento de demanda;

- **Resiliência:** o backend Express.js mantém registros de execução e logs de erro para identificação de falhas, possibilitando correções rápidas e facilitando a monitoração contínua do funcionamento do sistema;
- **Segurança:** as credenciais são armazenadas com hashing criptográfico, a autenticação é realizada via JWT, e entradas do usuário passam por validação, em conformidade com a Lei Geral de Proteção de Dados (LGPD) (Brasil, 2018) e com a especificação formal de tokens JWT (Jones; Bradley; Sakimura, 2015).

3.4 FERRAMENTAS TECNOLÓGICAS

3.4.1 Linguagens de Programação

As linguagens utilizadas no projeto foram escolhidas pela adequação às suas funções específicas dentro da arquitetura híbrida do sistema.

- **JavaScript (Node.js e React):** escolhida pela versatilidade, ampla adoção no desenvolvimento web, facilidade de integração entre frontend e backend e suporte nativo a operações assíncronas (Node.js Foundation, 2024; React Contributors, 2024);
- **Python (FastAPI):** utilizada no módulo de IA pela robustez em algoritmos complexos, disponibilidade de bibliotecas avançadas para NLP e aprendizado de máquina e excelente desempenho em tarefas de processamento textual (Python Software Foundation, 2024; Tiangolo, 2024).

3.4.2 Frameworks e Bibliotecas

- **Express.js:** utilizado no backend principal pela sua leveza, versatilidade e excelente desempenho em operações assíncronas, além da ampla compatibilidade com o ecossistema JavaScript (Node.js Foundation, 2024);
- **FastAPI:** empregado no módulo de IA devido à sua alta performance em APIs Python, suporte nativo a validação de dados e integração simples com bibliotecas de processamento de linguagem natural (Tiangolo, 2024);
- **Pandas:** utilizado para manipulação de dados e pré-processamento de textos, oferecendo alta compatibilidade com demais bibliotecas Python voltadas a análise e NLP (Pandas Development Team, 2024);
- **shadcn/ui:** adotado no frontend React para construção de interfaces visuais modernas, consistentes e com componentes reutilizáveis, acelerando o desenvolvimento das telas (shadcn, 2024);

- **PostgreSQL:** utilizado como banco de dados relacional principal do sistema, devido à sua maturidade, estabilidade e licença permissiva (PostgreSQL Global Development Group, 2024);
- **Outras bibliotecas:** serão adicionadas conforme a evolução do projeto, especialmente ferramentas complementares para interface, validação, análise textual e integração entre módulos.

3.4.3 Ferramentas de Desenvolvimento e Gestão

As ferramentas adotadas no desenvolvimento do *Agilis Licitações* foram escolhidas pela praticidade, integração com o fluxo de trabalho e suporte às tecnologias utilizadas no projeto.

- **Visual Studio Code:** ambiente de desenvolvimento utilizado para edição de código, organização do projeto e integração com extensões voltadas a JavaScript e Python (Visual Studio Code Team, 2024);
- **GitHub:** plataforma de versionamento e colaboração utilizada para armazenar o código-fonte, controlar alterações e manter o histórico de desenvolvimento (GitHub, 2024);
- **Node.js e npm:** utilizados para gerenciamento de dependências do backend Express.js e do frontend React (Node.js Foundation, 2024);
- **Python e pip:** empregados para gerenciar bibliotecas do módulo de IA desenvolvido em FastAPI (Python Software Foundation, 2024);
- **Insomnia:** ferramenta utilizada para teste, depuração e validação das APIs do backend e do módulo de IA durante o desenvolvimento (Insomnia, 2024).

3.4.4 Licenciamento

As tecnologias utilizadas no desenvolvimento do *Agilis Licitações* adotam licenças permissivas, permitindo modificação, distribuição e uso comercial sem grandes restrições.

- **JavaScript, Node.js, React, Express.js e shaden/ui:** licenciados sob **MIT**, permitindo uso livre e ampla flexibilidade no desenvolvimento;
- **FastAPI:** distribuído sob a licença **MIT**, favorecendo criação de APIs de forma aberta e permissiva;
- **Python:** licenciado sob a **Python Software Foundation License 2.0 (PSF)**;
- **Pandas:** distribuído sob a licença **BSD-3-Clause**;
- **PostgreSQL:** utiliza a **PostgreSQL License**, similar a licenças permissivas amplamente empregadas em software livre;

- **Insomnia:** licenciado sob **MIT**, permitindo uso livre como ferramenta de teste de APIs.

3.5 CONSIDERAÇÕES DE SEGURANÇA

3.5.1 Riscos Identificados

Os principais riscos associados ao funcionamento do sistema incluem:

- Vazamento de credenciais de usuário;
- Injeção de código por entradas malformadas;
- Envio de arquivos potencialmente maliciosos ou não suportados;
- Falhas de autenticação ou uso indevido de tokens;
- Instabilidade causada por uploads excessivamente grandes ou acima do limite operacional.

3.5.2 Medidas de Mitigação

Para reduzir os riscos identificados, o sistema adota as seguintes medidas:

- **Armazenamento seguro de credenciais**, utilizando hashing criptográfico;
- **Validação** dos dados de entrada em uploads e formulários;
- **Autenticação com tokens JWT**, garantindo controle de acesso seguro (Jones; Bradley; Sakimura, 2015);
- **Restrição de tipos de arquivo**, permitindo apenas **PDF** e **TXT**;
- **Limite de tamanho** de até **250 MB** por arquivo;
- **Limite de quantidade** de até **30 arquivos** por análise;

3.5.3 Normas e Boas Práticas Seguidas

O sistema segue diretrizes básicas de segurança recomendadas para aplicações web e serviços distribuídos, com foco no cumprimento da legislação vigente de proteção de dados:

- **Diretrizes essenciais da LGPD**, aplicadas exclusivamente ao tratamento de dados de login e autenticação (Brasil, 2018);
- **Separação entre módulos** (frontend, backend e módulo de IA), reduzindo exposição de serviços.

3.5.4 Responsabilidade Ética

Embora utilize inteligência artificial, o sistema opera exclusivamente sobre **documentos públicos**, não manipulando dados pessoais sensíveis. Dessa forma, mantém aderência a princípios éticos de privacidade, transparência e uso responsável de IA, em consonância com os fundamentos da LGPD (Brasil, 2018).

REFERÊNCIAS

- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software Architecture in Practice**. 3. ed. Boston: Addison-Wesley, 2012. Citado 3 vezes nas páginas 13, 14 e 19.
- BELTAGY, Iz; PETERS, Matthew; COHAN, Arman. Longformer: The long-document transformer. In: **arXiv preprint arXiv:2004.05150**. [S.l.: s.n.], 2020. Citado na página 12.
- BISHOP, Christopher M. **Pattern Recognition and Machine Learning**. New York: Springer, 2006. Citado na página 7.
- BRASIL. **Lei n. 13.709, de 14 de agosto de 2018: Lei Geral de Proteção de Dados Pessoais (LGPD)**. 2018. https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709.htm. Acesso em: 15 nov. 2025. Citado 3 vezes nas páginas 20, 22 e 23.
- BRASIL. **Lei n. 14.133, de 1º de abril de 2021: Nova Lei de Licitações e Contratos**. 2021. https://www.planalto.gov.br/ccivil_03/_ato2019-2022/2021/lei/L14133.htm. Acesso em: 15 nov. 2025. Citado 4 vezes nas páginas 4, 6, 7 e 11.
- BRECK, Eric et al. **Reliable Machine Learning: Applying SRE Principles to ML Systems**. Sebastopol: O'Reilly Media, 2022. Citado na página 8.
- BROWN, Simon. **The C4 Model for Visualising Software Architecture**. 2018. <https://c4model.com>. Acesso em: 15 nov. 2025. Citado na página 15.
- BROWN, Tom B. et al. Language models are few-shot learners. **NeurIPS**, 2020. Citado na página 13.
- CHALKIDIS, Ilias et al. Legal-bert: The muppets straight out of law school. **arXiv preprint**, 2020. Citado 6 vezes nas páginas 4, 5, 6, 7, 10 e 12.
- DEVLIN, Jacob et al. Bert: Pre-training of deep bidirectional transformers for language understanding. **Proceedings of NAACL-HLT**, p. 4171–4186, 2019. Citado 3 vezes nas páginas 4, 10 e 12.
- FOWLER, Martin. **UML Distilled: A Brief Guide to the Standard Object Modeling Language**. 3. ed. Boston: Addison-Wesley, 2004. Citado na página 10.
- GitHub. **GitHub Documentation**. 2024. <https://docs.github.com>. Acesso em: 15 nov. 2025. Citado na página 21.
- GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016. Citado na página 4.
- HASSAN, Samhaa R.; BAUMGARTNER, Robert. Automatic parsing of pdf documents: A survey. In: **Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)**. [S.l.: s.n.], 2016. Citado na página 12.
- HENDRYCKS, Dan et al. Lexnlp: Natural language processing for legal texts. **Journal of AI and Law**, 2020. Disponível em: <https://lexnlp.readthedocs.io>. Citado 3 vezes nas páginas 5, 7 e 10.
- Insomnia. **Insomnia API Client Documentation**. 2024. <https://docs.insomnia.rest>. Acesso em: 15 nov. 2025. Citado na página 21.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **JSON Web Token (JWT)**. 2015. RFC 7519. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519>. Acesso em: 15 nov. 2025. Citado 2 vezes nas páginas 20 e 22.

JURAFSKY, Daniel; MARTIN, James H. **Speech and Language Processing**. 3. ed. New Jersey: Prentice Hall, 2023. Versão preliminar disponível online. Citado na página 4.

KIM, Soo; LEE, Ji-Hoon; PARK, Sungho. Limitations of rule-based information extraction in heterogeneous documents. In: **Proceedings of the International Conference on Document Analysis**. [S.l.: s.n.], 2019. p. 55–62. Citado 3 vezes nas páginas 4, 6 e 7.

LAKSHMANAN, Valliappa; ROBINSON, Sara; MUNN, Michael. **Machine Learning Design Patterns: Solutions to Common Challenges in Data Preparation, Model Building, and MLOps**. Sebastopol: O'Reilly Media, 2020. Citado 2 vezes nas páginas 7 e 8.

MANNING, Christopher D.; SCHÜTZ, Hinrich. **Foundations of Statistical Natural Language Processing**. Cambridge: MIT Press, 1999. Citado 2 vezes nas páginas 7 e 8.

Node.js Foundation. **Node.js Documentation**. 2024. <https://nodejs.org>. Acesso em: 15 nov. 2025. Citado 5 vezes nas páginas 10, 13, 19, 20 e 21.

Pandas Development Team. **pandas Documentation**. 2024. <https://pandas.pydata.org>. Acesso em: 15 nov. 2025. Citado 2 vezes nas páginas 19 e 20.

PostgreSQL Global Development Group. **PostgreSQL Documentation**. 2024. <https://www.postgresql.org/docs>. Acesso em: 15 nov. 2025. Citado 2 vezes nas páginas 14 e 21.

Python Software Foundation. **Python Documentation**. 2024. <https://docs.python.org/3>. Acesso em: 15 nov. 2025. Citado 3 vezes nas páginas 19, 20 e 21.

PÚBLICAS, PNCP Portal Nacional de Contratações. **Estatísticas de Licitações e Contratações Públicas**. 2024. <https://www.gov.br/pncp>. Acesso em: 15 nov. 2025. Citado 3 vezes nas páginas 5, 6 e 7.

React Contributors. **React Documentation**. 2024. <https://react.dev>. Acesso em: 15 nov. 2025. Citado 3 vezes nas páginas 13, 19 e 20.

RUSSELL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 4. ed. London: Pearson, 2020. Citado 2 vezes nas páginas 4 e 6.

SHADCN. **shadcn/ui Documentation**. 2024. <https://ui.shadcn.com>. Acesso em: 15 nov. 2025. Citado 2 vezes nas páginas 19 e 20.

SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson, 2011. Citado na página 9.

(TCU), Tribunal de Contas da União. **Relatório de Auditoria sobre Padronização de Editais e Processos de Contratação**. 2022. <https://www.tcu.gov.br>. Acesso em: 15 nov. 2025. Citado 3 vezes nas páginas 4, 6 e 7.

TIANGOLO, Sebastián. **FastAPI Documentation**. 2024. <https://fastapi.tiangolo.com>. Acesso em: 15 nov. 2025. Citado 3 vezes nas páginas 10, 13 e 20.

UTIYAMA, Masao; ISAHARA, Hitoshi. A statistical model for domain-independent text segmentation. In: **ACL**. [S.l.: s.n.], 2001. Citado na página 12.

Visual Studio Code Team. **Visual Studio Code Documentation**. 2024. <https://code.visualstudio.com/docs>. Acesso em: 15 nov. 2025. Citado na página 21.

WOLF, Thomas et al. Transformers: State-of-the-art natural language processing. **Proceedings of EMNLP**, 2020. Citado na página 12.

APÊNDICE A – CONJUNTO DE LICITAÇÕES UTILIZADAS NA AVALIAÇÃO

A seguir são apresentadas as licitações utilizadas para avaliação das métricas do sistema *Agilis Licitações*, conforme definido nos requisitos não funcionais. Cada item inclui o número da licitação, UASG correspondente, órgão responsável e o link oficial para consulta.

1. **Licitação:** 900062025

UASG: 160121

Órgão: Hospital Geral de Juiz de Fora

Link: <https://cnetmobile.estaleiro.serpro.gov.br/comprasnet-web/public/compras/acompanhamento-compra?compra=16012105900062025>

2. **Licitação:** 901042025

UASG: 985833

Órgão: Prefeitura Municipal de Duque de Caxias — RJ

Link: <https://cnetmobile.estaleiro.serpro.gov.br/comprasnet-web/public/compras/acompanhamento-compra?compra=98583305901042025>

3. **Licitação:** 908222025

UASG: 92301

Órgão: ESP — Hospital das Clínicas de São Paulo

Link: <https://cnetmobile.estaleiro.serpro.gov.br/comprasnet-web/public/compras/acompanhamento-compra?compra=09230105908222025>

4. **Licitação:** 3342025

UASG: 927502

Órgão: Fundo Estadual de Saúde de Rondônia

Link: <https://pncp.gov.br/app/editais/00733062000102/2025/356>

5. **Licitação:** 900182025

UASG: 160039

Órgão: Hospital Geral de Salvador

Link: <https://cnetmobile.estaleiro.serpro.gov.br/comprasnet-web/public/compras/acompanhamento-compra?compra=16003905900182025>

Fonte: Elaborado pelo autor (2025).